Calculadora de lançamentos oblíquos

Essa é uma calculadora de movimentos parabólicos em C, com 6 funcionalidades:

- Calcular o tempo para chegar em uma posição
- Calcular a posição em um determinado tempo
- Calcular a velocidade em uma determinada posição
- Calcular a velocidade em um determinado tempo
- Calcular o ponto de encontro entre dois movimentos
- Calcular o ponto mais alto

Vale notar que, pelo funcionamento dos computadores, a calculadora, da maneira simples apresentada, terá problemas com valores com dígitos decimais – para resolver isso, teríamos que trabalhar com arredondamentos, tolerando pequenas variações entre resultados; o código apresentado aqui, para não ser muito técnico, não tem essas considerações, mas um código com tais considerações pode ser acessado no final do relatório.

Começamos definindo algumas constantes que serão utilizadas durante o programa: pi e a aceleração da gravidade.

```
#define PI 3.14159
#define gravidade 9.81
```

Vamos criar uma função para calcular o Bhaskara de equações do tipo $ax^2+bx+c = 0$:

```
double *bhaskara(double a, double b, double c) {
    // Caso linear: a == 0
    if (a==0) {
        if (b==0) {
            return NULL; // sem solução
        }
        double *raizes = malloc(2*sizeof(double));
        double root = -c/b; // bx + c = 0 -> x = -c/b
        raizes[0] = root;
        raizes[1] = root;
        return raizes;
    }

    double delta = b*b-4*a*c;
```

```
    double *raizes = malloc(2*sizeof(double));

    if (delta < 0) {
        printf("Não existem raízes reais.\n");
        free(raizes);
        return NULL;
    } else if (delta==0) {
        float root = -b/(2*a);
        raizes[0] = root;
        raizes[1] = root;
    } else {
        raizes[0] = (-b + sqrt(delta))/(2*a);
        raizes[1] = (-b - sqrt(delta))/(2*a);
    }
    return raizes;
}
```

Perceba, ainda, que essa função aceita equações do tipo bx + c: trata-se de uma escolha minha para facilitar as contas, evitando criar verificações para esse caso específico em cada parte do código que faz uso dessa função. O resto da função funciona como o esperado.

Agora, na parte principal do código, vamos definir:

```
int main (void) {
    int escolha = 7;
    printf("Todos os valores devem ser inseridos em unidades do SI (m, s, m/s,
m/s²)\n");
    double Sox,Soy,Sx,Sy,Vo,ax,t,theta,Vox,Voy;
```

O valor inteiro “escolha” é utilizado posteriormente para que o usuário escolha entre as funcionalidades apresentadas anteriormente. Além disso, informamos o usuário logo no início de que o programa espera valores no SI – os valores números apresentados permaneceriam adequados se unidades diferentes forem inseridas, mas não bateria com os “prints” apresentados ao usuário, como veremos em seguida.

Definimos, ainda, algumas variáveis que serão utilizadas em nosso código: Sox (posição inicial em x), Soy, Sx (posição final/desejada em x), Sy, Vo (velocidade inicial), ax (aceleração no eixo x), t (tempo), theta (ângulo em graus do lançamento), Vox e Voy.

```
do {

        switch(escolha) {
```

```
            case 7:
                printf("Digite a posição inicial no eixo x: ");
                scanf("%lf",&Sox);
                printf("Digite a posição inicial no eixo y: ");
                scanf("%lf",&Soy);
                printf("Digite a velocidade inicial: ");
                scanf("%lf",&Vo);
                printf("Digite a resistência do ar (em módulo): ");
                scanf("%lf",&ax);
                printf("Digite o ângulo de inclinação (em graus): ");
                scanf("%lf",&theta);
                theta *= (PI/180);
                Vox = Vo*cos(theta);
                Voy = Vo*sin(theta);
                break;
```

No começo do programa (ou sempre que a opção 7 for escolhida), o usuário deverá inserir algumas informações básicas para o programa funcionar. Após isso, theta é convertido para radianos e Vox e Voy são calculados a partir de Vo e theta. Note que a aceleração no eixo x citada anteriormente é pensada como sendo a resistência do ar – entretanto, se o usuário inserir o valor negativo, o código funcionará como se houvesse uma aceleração positiva no eixo x no movimento. Após isso, o código prosseguirá para:

```
        printf("\n\n");

        printf("Escolha uma opção:\n");
        printf("1 - Calcular o tempo para chegar em uma posição\n");
        printf("2 - Calcular a posição em um determinado tempo\n");
        printf("3 - Calcular a velocidade em uma determinada posição\n");
        printf("4 - Calcular a velocidade em um determinado tempo\n");
        printf("5 - Calcular o ponto de encontro entre dois movimentos\n");
        printf("6 - Calcular o ponto mais alto\n");
        printf("7 - Digitar novos valores\n");
        printf("0 - Sair\n");

        scanf("%d",&escolha);
    }while(escolha != 0);
}
```

Após o usuário inserir as informações importantes, ele será perguntado que conta deseja fazer. Vamos analisá-las uma a uma.

```
            case 1:
                printf("Digite a posição desejada no eixo x: ");
                scanf("%lf", &Sx);
                printf("Digite a posição desejada no eixo y: ");
                scanf("%lf", &Sy);
```

```
                // Calcula tempos pelo eixo X
                double *raizesX = bhaskara(-ax/2,Vox,Sox-Sx);
                if (raizesX == NULL) {
                    printf("Não foi possível calcular o tempo para a posição x
desejada.\n");
                    break;
                }
                double t1 = raizesX[0];
                double t2 = raizesX[1];
                free(raizesX);

                int found = 0;
                double t_sol = -1;

                // Verifica se algum tempo de X bate com Sy
                double cand[2] = {t1,t2};
                for (int i=0; i<2; ++i) {
                    float ti = cand[i];
                    if (ti<0) {
                        continue;
                    }
                    double yti = Soy + Voy*ti - gravidade*ti*ti/2;
                    if (yti==Sy) {
                        found=1;
                        t_sol=ti;
                        break;
                    }
                }

                if (found) {
                    printf("O tempo para chegar na posição desejada é: %.2f
segundos\n", t_sol);
                } else {
                    printf("Não foi possível encontrar um tempo válido para a
posição desejada.\n\n");
                }
                break;
```

Se o usuário quiser calcular o tempo para chegar em uma determinada posição, ele deverá informar a posição exigida (Sx e Sy). Após isso, calculamos os possíveis tempos através de $S = So + Vot + at^2/2$, considerando o eixo x. Se for possível encontrar ao menos um tempo real, nós verificamos, em seguida, se algum dos tempos calculados condiz com a trajetória do corpo em y comparando com a posição em y que se quer obter (Sy) para esse determinado tempo.

Se a trajetória em y e em x bater com a posição desejada em y e em x pelo usuário, imprimimos o tempo, como pedido; do contrário, informamos que a posição desejada não é alcançada durante o movimento.

```
        case 2:
                printf("Digite o tempo desejado: ");
                scanf("%lf",&t);

                Sx = Sox+Vox*t+(-ax*t*t)/2;
                Sy = Soy+Voy*t+(-gravidade*t*t)/2;
                printf("A posição no tempo %.2f segundos é: (%.2f,
%.2f)\n\n",t,Sx,Sy);

                break;
```

Já caso o usuário queira calcular a posição em um determinado tempo, podemos resolver isso de maneira muito mais simples: basta substituir os valores na fórmula $S = So + Vot + at^2/2$, tanto para x, quanto para y. Vale notar que não estipulamos um "limite" em y=0, isso é, para tempos grandes, a função retornará y's negativos. Se fosse desejado limitar para y=0, poderíamos fazer uma verificação simples.

```
        case 3:
                printf("Digite a posição desejada no eixo x: ");
                scanf("%lf",&Sx);
                printf("Digite a posição desejada no eixo y: ");
                scanf("%lf",&Sy);

                double Vx, Vy, V;
                double t_sol = -1;

                // Resolve pelo eixo X e valida com o eixo Y
                double *raizesX = bhaskara(-ax/2,Vox,Sox-Sx);
                if (raizesX != NULL) {
                    double t1 = raizesX[0];
                    double t2 = raizesX[1];
                    free(raizesX);

                    double cand[2] = {t1,t2};
                    for (int i=0; i<2; ++i) {
                        double ti = cand[i];
                        if (ti < 0) {
                            continue;
```

```
                        }
                        double yti = Soy + Voy*ti - gravidade*ti*ti/2;
                        if (yti==Sy) {
                            t_sol = ti;
                            break;
                        }
                    }
                }

                if (t_sol < 0) {
                    printf("Não foi possível determinar um tempo que satisfaça
(Sx, Sy) simultaneamente.\n");
                    break;
                }

                Vx = Vox+(-ax)*t_sol;
                Vy = Voy+(-gravidade)*t_sol;
                V = sqrt(Vx*Vx+Vy*Vy);

                printf("A velocidade na posição (%.2f, %.2f) é: %.2f m/s.\nNo
eixo x, temos: %.2f m/s; no eixo y, temos: %.2f m/s\n\n",Sx,Sy,V,Vx,Vy);
                break;
```

Nessa situação, o usuário informa uma posição e obtém a velocidade. Aqui, encontramos o tempo da mesma maneira que no caso 1 (como a verificação se (Sx,Sy) realmente ocorre no lançamento é feita mais de uma vez com a mesma lógica, o melhor seria ter feito uma função para melhorar a legibilidade do código). Se não encontramos (Sx,Sy) na trajetória, retornamos informando isso ao usuário.

Em seguida, calculamos a velocidade instantânea em cada eixo por V(t) = Vo+a*t. Após isso, obtemos a rapidez do objeto no instante por $\sqrt{Vx^2 + Vy^2}$.

```
            case 4:
                printf("Digite o tempo desejado: ");
                scanf("%lf",&t);
                double Vx2,Vy2,V2;
                Vx2 = Vox+(-ax)*t;
                Vy2 = Voy+(-gravidade)*t;

                V2 = sqrt(Vx2*Vx2 + Vy2*Vy2);
                printf("A velocidade no tempo %.2f segundos é: %.2f m/s.\nNo
eixo x, temos: %.2f m/s; no eixo y, temos: %.2f m/s\n\n",t,V2,Vx2,Vy2);
                break;
```

Aqui, o usuário simplesmente insere o tempo e recebe a velocidade do lançamento nesse tempo. Isso é feito assim como no caso anterior: V(t) = Vo+a*t.

```
        case 5:
                printf("Digite as informações do segundo movimento:\n");
                double Sox2, Soy2, Vo2, ax2, theta2, Vox2, Voy2;
                printf("Digite a posição inicial no eixo x: ");
                scanf("%lf", &Sox2);
                printf("Digite a posição inicial no eixo y: ");
                scanf("%lf", &Soy2);
                printf("Digite a velocidade inicial: ");
                scanf("%lf", &Vo2);
                printf("Digite a resistência do ar (em módulo): ");
                scanf("%lf", &ax2);
                printf("Digite o ângulo de inclinação (em graus): ");
                scanf("%lf", &theta2);
                theta2 *= (PI/180.0);
                Vox2 = Vo2 * cos(theta2);
                Voy2 = Vo2 * sin(theta2);

                double *raizes2 = bhaskara((ax2-ax)/2,(Vox-Vox2),(Sox-Sox2));
                if (raizes2 == NULL) {
                    printf("Não foi possível calcular o tempo para a posição x
desejada.\n");
                    break;
                }
                double t12 = raizes2[0];
                double t22 = raizes2[1];
                raizes2 = bhaskara(0,(Voy-Voy2),(Soy-Soy2));
                if (raizes2 == NULL) {
                    printf("Não foi possível calcular o tempo para a posição y
desejada.\n");
                    break;
                }
                double t32 = raizes2[0];
                double t42 = raizes2[1];
                free(raizes2);

                if (t12 >= 0 && (t12==t32 || t12==t42)) {
                    printf("O tempo para o encontro dos dois movimentos é:
%.2f segundos\n",t12);
                } else if (t22 >= 0 && (t22==t32 || t22==t42)) {
                    printf("O tempo para o encontro dos dois movimentos é:
%.2f segundos\n",t22);
                } else {
                    printf("Não foi possível encontrar um tempo válido para o
encontro dos dois movimentos.\n\n");
```

```
            }

            break;
```

Nesse caso mais complexo, o usuário deseja obter o momento em que dois corpos, cada um em um lançamento oblíquo, colidem. Então, pedimos para que o usuário informe todos os dados do segundo movimento, exatamente como para o primeiro lançamento.

Após isso, igualamos as equações $S = So + Vot + at^2/2$ de cada um dos movimentos, já que o S desejado é o mesmo para ambos (se queremos quando houve colisão, ambos objetos devem estar no mesmo ponto).

Por meio dessa igualdade, são formadas equações em função do tempo que, ao serem resolvidas, indicam os instantes possíveis de encontro dos corpos. Com os dados informados, o programa verifica todas as soluções encontradas, descartando tempos negativos e casos em que os tempos dos dois movimentos não coincidem, e aponta o resultado final para o usuário.

```
        case 6:
            if (Voy <= 0) {
                printf("O movimento não possui subida (Voy <= 0).\n");
                break;
            }
            // Tempo até o ponto mais alto: Vy = 0
            t = Voy/gravidade;

            // Posição nesse instante
            Sx = Sox + Vox*t +(-ax*t*t)/2;
            Sy = Soy + Voy*t - (gravidade*t*t)/2;

            printf("Ponto mais alto atingido em %.2f segundos.\n",t);
            printf("Posição no ponto mais alto: (%.2f, %.2f)\n",Sx,Sy);
            break;
```

Para o último caso, retornamos ao usuário a posição e o instanto do ponto mais alto da trajetória. O momento em que isso ocorre é obtido por $t = \frac{Voy}{gravidade}$, uma vez que $Voy = Vo*sen(\theta)$ e $Vy(t) = Voy - g*t$. No ponto mais alto, temos $Vy(t) = 0 \rightarrow g*t = Voy \rightarrow t = \frac{Voy}{g}$. Então, calculamos a posição com esse tempo e informamos ao usuário.

Finalmente, terminamos o código com:

```
            case 0:
                return 1;
                break;
            default:
                printf("Opção inválida. Tente novamente.\n");
                break;
        }
        printf("\n\n");
```

Isso é, finalizamos o programa se o usuário digitar “0” e pedimos para ele reinserir a opção escolhida caso não escolha nenhuma dentre as possíveis.

Vamos ver esse código rodando em alguns casos (o texto está sem acentuação):

```
Todos os valores devem ser inseridos em unidades do SI (m, s, m/s, m/s┬)
Digite a posi├º├úo inicial no eixo x: 0
Digite a posi├º├úo inicial no eixo y: 0
Digite a velocidade inicial: 10
Digite a resist├¬ncia do ar (em m├│dulo): 0
Digite o ├óngulo de inclina├º├úo (em graus): 45
```

```
6
Ponto mais alto atingido em 0.72 segundos.
Posi├º├úo no ponto mais alto: (5.10, 2.55)
```

```
1
Digite a posi├º├úo desejada no eixo x: 5.10
Digite a posi├º├úo desejada no eixo y: 2.55
O tempo para chegar na posi├º├úo desejada ├⌐: 0.72 segundos
```

```
3
Digite a posi├º├úo desejada no eixo x: 5.10
Digite a posi├º├úo desejada no eixo y: 2.55
A velocidade na posi├º├úo (5.10, 2.55) ├⌐: 7.07 m/s.
No eixo x, temos: 7.07 m/s; no eixo y, temos: -0.00 m/s
```

```
4
Digite o tempo desejado: 0.72
A velocidade no tempo 0.72 segundos ├⌐: 7.07 m/s.
No eixo x, temos: 7.07 m/s; no eixo y, temos: 0.01 m/s
```

Note que, nesse últimos 2 exemplos, alguns problemas aparecem justamente devido à precisão de decimais: 0 está como negativo e, mesmo 3 e 4 fazendo a mesma coisa para obter a velocidade no eixo y, um dos casos retornou 0.01m/s e o outro -0m/s.

```
5
Digite as informa├º├┤es do segundo movimento:
Digite a posi├º├úo inicial no eixo x: 10.2
Digite a posi├º├úo inicial no eixo y: 0
Digite a velocidade inicial: -10
Digite a resist├¬ncia do ar (em m├│dulo): 0
Digite o ├óngulo de inclina├º├úo (em graus): -45
O tempo para o encontro dos dois movimentos ├⌐: 0.72 segundos
```

O código completo pode ser acessado nesse arquivo do Onedrive. Como dito, por questão de precisão de pontos decimais, esse código pode apresentar problemas. Então, um outro código que faz uso de aproximações pode ser acessado aqui.